

Московская предпрофессиональная олимпиада школьников

Профиль «Информационные технологии»

ДОКУМЕНТАЦИЯ

Командный кейс № 3

«Анализ поступления»

Команда разработчиков:

Артём Колеров,
Динар Рахматуллин,
Лев Шаповалов,
Марк Шаповалов,
Максим Жуланов

Москва, 2025-2026

СОДЕРЖАНИЕ

1. Обоснование выбора языка программирования	3
1.1. Выбор языков программирования	3
1.2. Используемые технологии	3
1.3. Преимущества архитектуры	4
2. Структурная и функциональная схемы	5
2.1. Архитектура системы	5
2.2. Взаимодействие компонентов	5
2.3. Структура проекта	6
3. Блок-схема основного алгоритма	7
3.1. Алгоритм Deferred Acceptance	7
3.2. Правила сортировки.....	7
3.3. Код алгоритма.....	8
4. Описание типа хранения данных	9
4.1. Причины выбора	9
4.2. Логика обновления данных	9
4.3. Структура данных абитуриента	10
4.4. Образовательные программы	10
4.5. Формат CSV-файлов	11
5. Скриншоты интерфейса	12
6. Ссылка на репозиторий	14

1. Обоснование выбора языка программирования

1.1. Выбор языков программирования

Для реализации проекта была выбрана двухкомпонентная архитектура с использованием двух языков программирования:

Python 3.12 — аналитический движок

Python + FastAPI: подходит для обработки данных, построения статистики, графики динамики конкурсных списков и генерации PDF-отчетов. FastAPI обеспечивает HTTP-взаимодействие между веб-приложением на .NET и аналитическим модулем. Это считается эталонным подходом в аналитике данных

C# / ASP.NET Core MVC — веб-приложение

C# + ASP.NET Core MVC: быстрая разработка форм и API, встроенная DI/Model Binding/Validation, хорошая поддержка EF Core для работы с БД.

1.2. Используемые технологии и библиотеки

Компонент	Технология	Назначение
Backend API	FastAPI + Uvicorn	REST API для аналитики
Web Framework	ASP.NET Core MVC	Веб-интерфейс пользователя
Визуализация	matplotlib	Построение графиков
PDF генерация	ReportLab	Формирование отчётов
Frontend	HTML, CSS(кастомная вёрстка)	CSS-фреймворк для UI
Формат данных	CSV, JSON	Хранение и обмен данными

1.3. Преимущества выбранной архитектуры

- Разделение ответственности: Python отвечает за сложные вычисления, C# — за пользовательский интерфейс
- Масштабируемость: каждый компонент можно масштабировать независимо
- Гибкость: Python API может использоваться и другими клиентами (мобильные приложения, другие сервисы)
- Тестируемость: каждый компонент можно тестировать изолированно
- Современный стек: используются актуальные версии технологий (Python 3.12, .NET 10)

2. Структурная и функциональная схемы программного продукта

2.1. Архитектура системы

Система AdmissionAnalysis построена по трёхуровневой архитектуре:

Уровень	Технология	Описание
Презентационный (Frontend)	ASP.NET Core MVC, jQuery, чистый HTML/CSS	Веб-интерфейс для пользователя. Отображение данных, навигация, формы загрузки.
Бизнес-логика (Backend API)	Python FastAPI, Uvicorn	REST API для аналитики. Расчёт проходных баллов, генерация отчётов.
Данные (Storage)	Файловая система: CSV, JSON, PDF	Хранение конкурсных списков, результатов анализа и отчётов.

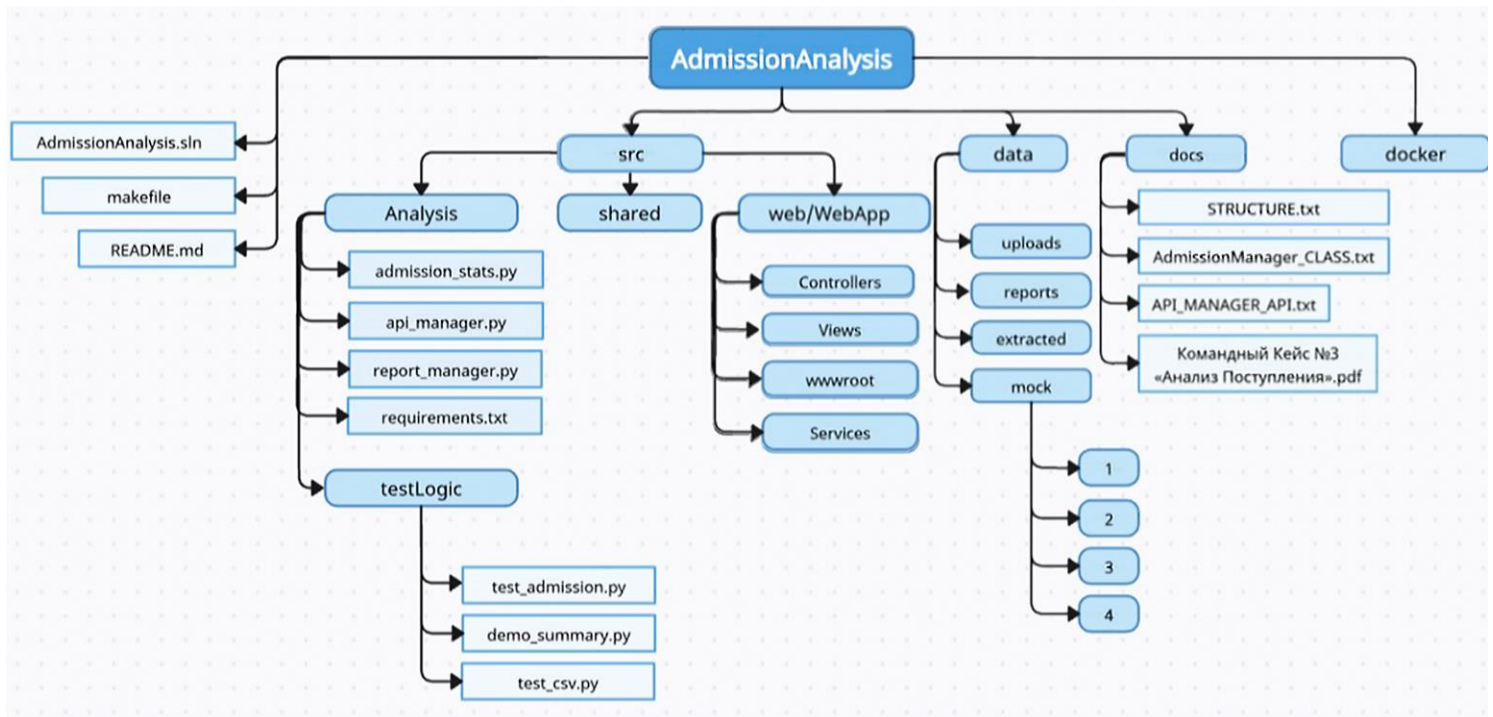
2.2. Взаимодействие между компонентами

1. Пользователь открывает веб-приложение ASP.NET (localhost:5002).
2. ASP.NET отправляет HTTP-запросы к Python API (localhost:8000).
3. Python API загружает CSV из data/mock/ или data/uploads/.
4. AdmissionManager выполняет алгоритм Deferred Acceptance.
5. Результаты анализа сохраняются в data/reports/{дата}/stats.json.
6. ReportManager формирует PDF с графиками и таблицами.

Ключевые компоненты системы:

Компонент	Назначение
AdmissionController.cs	Главная страница поступления, загрузка CSV, импорт в БД
api_manager.py	REST API endpoints: /api/analyze, /api/generate_pdf_report
admission_stats.py	Класс AdmissionManager — алгоритм распределения
report_manager.py	Класс ReportManager — генерация PDF с графиками
data/mock/	Тестовые CSV-файлы конкурсных списков (4 дня)
data/reports/	Результаты анализа (stats.json) и PDF-отчёты
AnalyzeController.cs	API анализа (день/все дни), вызов FastAPI, возврат JSON
StatisticsController.cs	Правый столбец: Отображение статистики зачисления, работа с отчётами (stats.json, PDF)
VisualizeController.cs	Визуализация данных, выдача из БД в JSON для графиков

2.3. Структура проекта



3. Блок-схема работы основного алгоритма

3.1. Алгоритм Deferred Acceptance (отложенного принятия)

Система использует алгоритм Deferred Acceptance для справедливого распределения абитуриентов по образовательным программам с учётом приоритетов заявлений.

3.2. Правила сортировки абитуриентов

При конкурсном отборе применяется двухуровневая сортировка:

1. Первичный критерий: сумма баллов (total) — по убыванию
2. Вторичный критерий: ID абитуриента — по возрастанию (при равных баллах)

3.3. Псевдо код алгоритма

```
# Алгоритм Deferred Acceptance
programs = list[Any](self.capacity.keys())
next_priority = {aid: 1 for aid in candidates}
queue = list[Any](candidates.keys())
tentative = {p: [] for p in programs}
enrolled_program = {}

while queue:
    aid = queue.pop(0)
    priority = next_priority[aid]
    applicant = candidates[aid]

    # Ищем программу с таким приоритетом
    target = None
    for prog, prio in applicant['priorities'].items():
        if prio == priority:
            target = prog
            break

    if target is None:
        if priority < 4:
            next_priority[aid] = priority + 1
            queue.append(aid)
            continue

    next_priority[aid] = priority + 1
    tentative[target].append(applicant)
    tentative[target].sort(key=sort_key)

    cap = self.capacity.get(target, 0)
    if len(tentative[target]) > cap:
        rejected = tentative[target][cap:]
        tentative[target] = tentative[target][:cap]
        for r in rejected:
            if next_priority[r['id']] <= 4:
                queue.append(r['id'])
```

3.4. Особенности алгоритма

- Каждый абитуриент зачисляется максимум в одну программу
- Учитываются только абитуриенты с согласием (`consent=true`)
- Зачисленный на высокий приоритет не конкурирует на низкие приоритеты
- Гарантия завершения за $O(n)$ операций, где n — число абитуриентов
- Алгоритм стабилен: результат не зависит от порядка обработки

4. Описание особенностей и аргументация выбранного типа СУБД

4.1. Описание особенностей и аргументация выбранного типа СУБД

SQLite (через EF Core): требования ТЗ не фиксируют тип БД; SQLite лёгкая, встраиваемая, не требует отдельного сервера, достаточно быстра для объёмов кейса (до ≈ 5000 записей на день $\times 4$ дня ≈ 20000). Позволяет атомарные транзакции (`delete/insert/update`) и переносима между окружениями. При росте нагрузки можно безболезненно перейти на PostgreSQL, оставив EF Core. За счёт локального хранения данных и транзакционного обновления обеспечивается возможность соблюдения ограничений ТЗ по времени загрузки конкурсных списков (≤ 5 с) и перестроения визуализаций (≤ 3 с)

4.2. Схема базы данных

1. Одна таблица: `ApplicantsCurrent` (DbSet в `AdmissionContext`).
2. Назначение: хранение текущего состояния списков абитуриентов по образовательным программам (данные, загружаемые через веб-интерфейс из CSV).
3. Первичный ключ: составной (`Program, ApplicantId`) — задаётся в `OnModelCreating` в `ApplicantContext`.
4. Поля таблицы `ApplicantsCurrent`:

Название поля	Тип данных	Описание
ApplicantId	int	ID абитуриента (из CSV)
Program	int (enum)	Код программы: РМ, IVT, ITSS, IB (хранится как int)
Consent	bool	Согласие на зачисление (true/false)
Priority	int	Приоритет заявления (1–4)
PhysicsIct	int	Балл по физике/ИКТ (0–100)
Russian	int	Балл по русскому языку (0–100)
Math	int	Балл по математике (0–100)
Individual	int	Балл за индивидуальные достижения (0–10)
Total	int	Сумма баллов (0–310)

4.3. Образовательные программы

Код	Название	Количество мест
pm	Прикладная математика	40
ivt	Информатика и вычислительная техника	50
itss	Инфокоммуникационные технологии и системы связи	30
ib	Информационная безопасность	20

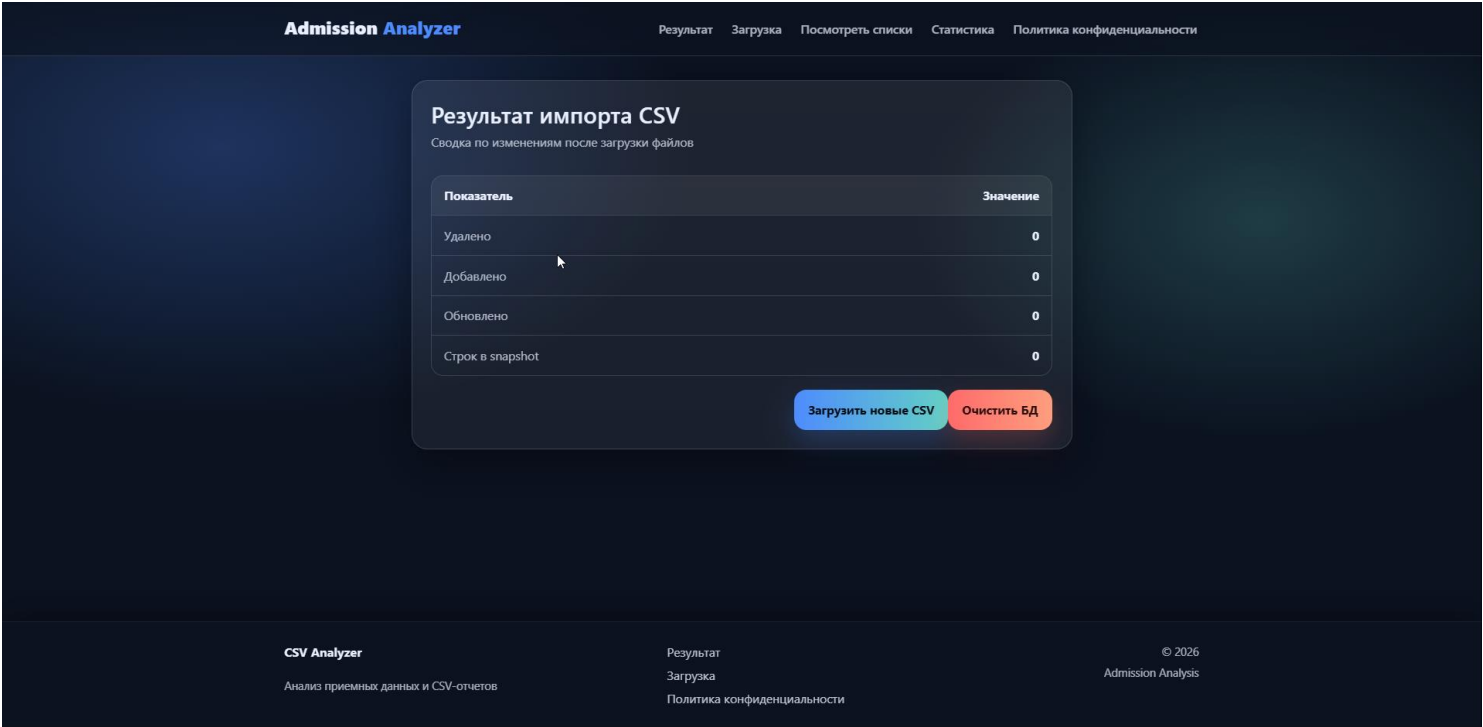
Всего: 140 бюджетных мест

4.4. Структура CSV-файла

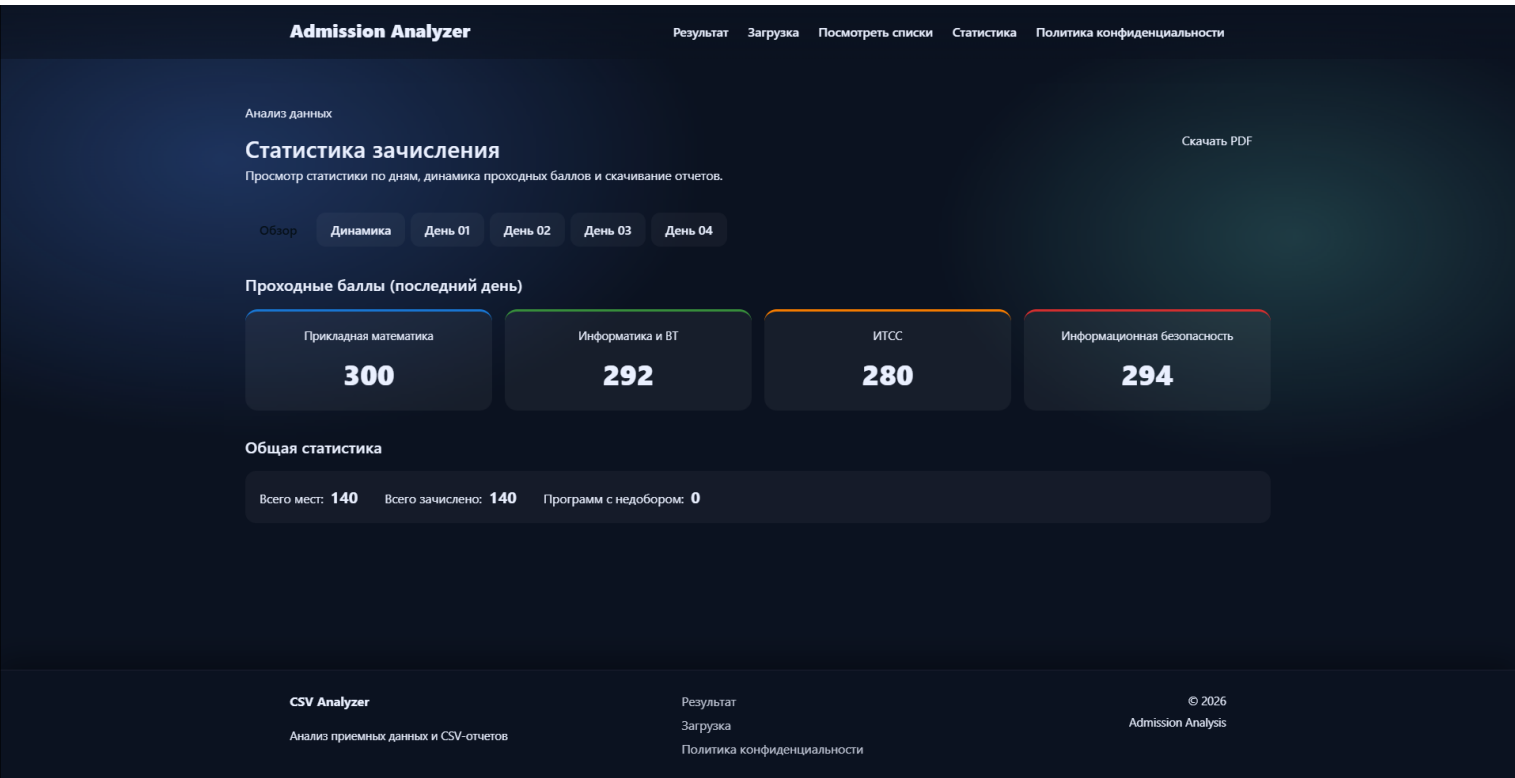
```
id;consent;priority;physics_ict;russian;math;individual;total
211;1;3;72;58;78;7;215
212;1;4;86;100;100;7;293
213;1;2;84;80;87;2;253
214;1;4;84;88;75;9;256
215;1;3;79;87;77;9;252
216;1;4;74;72;59;4;209
217;1;3;57;71;74;0;202
218;1;4;63;81;67;3;214
219;0;1;74;65;69;4;212
220;0;3;90;100;94;3;287
221;0;1;90;89;79;5;263
222;0;4;75;74;54;7;210
223;0;1;75;78;81;5;239
224;0;2;91;95;100;10;296
```

5. Скриншоты графического интерфейса и примеры отчётов

5.1. Главная страница веб-приложения



5.2. Страница анализа данных



5.3. Пример PDF-отчёта

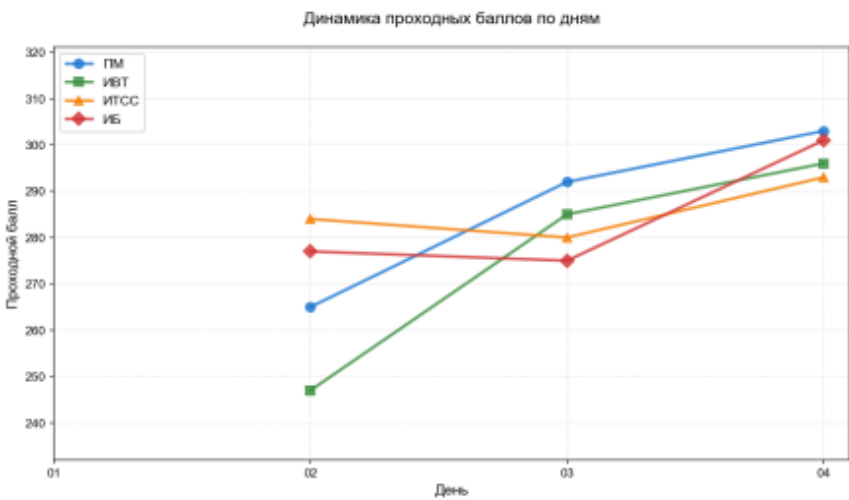
ОТЧЕТ О ЗАЧИСЛЕНИИ АБИТУРИЕНТОВ

Дата формирования: 02.01.2026 22:30:40
Период анализа: дни 01-04 (4 дней)
Данные на дату: 04

ПРОХОДНЫЕ БАЛЛЫ

Программа	Проходной балл
Прикладная математика	303
Информатика и вычислительная техника	296
Инфокоммуникационные технологии и системы связи	293
Информационная безопасность	301

ДИНАМИКА ПРОХОДНЫХ БАЛЛОВ



СПИСКИ ЗАЧИСЛЕННЫХ АБИТУРИЕНТОВ

Прикладная математика (40 чел.)

№	ID абитуриента	Сумма баллов
1	213	310
2	1624	310
3	1709	310
4	178	309
5	397	309
6	506	309
7	561	309
8	801	309
9	1773	309
10	370	308
11	538	308
12	647	308
13	696	308
14	829	308
15	1295	308
16	698	307
17	1388	307
18	1463	307
19	1765	307
20	422	306
21	1683	306
22	98	305
23	578	305
24	910	305
25	1363	305
26	1816	305
27	1823	305
28	1839	305
29	85	304
30	218	304
31	739	304
32	1067	304
33	1205	304
34	1229	304
35	1334	304
36	1346	304

34	163	298
35	331	298
36	1262	298
37	1342	298
38	1499	298
39	376	297
40	451	297
41	1112	297
42	1280	297
43	1398	297
44	1694	297
45	1899	297
46	73	296
47	182	296
48	618	296
49	963	296
50	964	296

Инфокоммуникационные технологии и системы связи (30 чел.)

№	ID абитуриента	Сумма баллов
1	1026	310
2	1895	305
3	1615	301
4	157	300
5	943	299
6	1722	297
7	1022	296
8	1047	296
9	1281	296
10	1519	296
11	354	295
12	848	295
13	1779	295
14	160	294
15	371	294
16	536	294
17	559	294
18	579	294

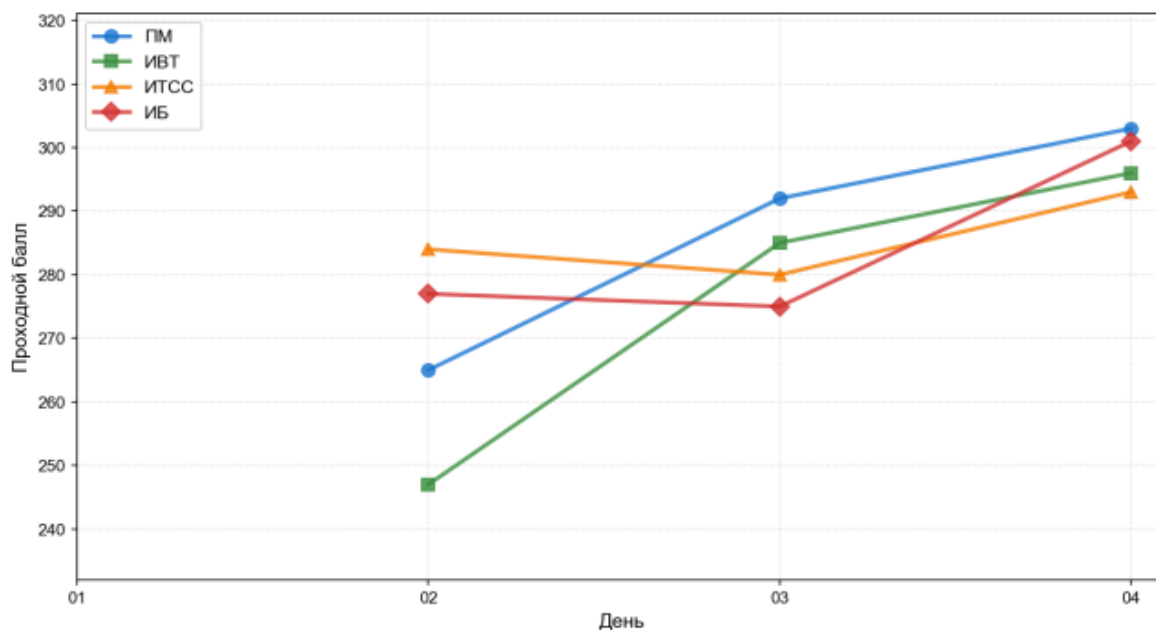
СТАТИСТИКА ПО ОБРАЗОВАТЕЛЬНЫМ ПРОГРАММАМ

Показатель	ПМ	ИВТ	ИТСС	ИБ
Общее кол-во заявлений	1067	1164	1065	1022
Количество мест на ОП	40	50	30	20
Кол-во заявлений 1-го приоритета	1067	213	41	102
Кол-во заявлений 2-го приоритета	0	74	112	920
Кол-во заявлений 3-го приоритета	0	877	49	0
Кол-во заявлений 4-го приоритета	0	0	863	0
Кол-во зачисленных 1-го приоритета	40	19	3	7
Кол-во зачисленных 2-го приоритета	0	8	3	13
Кол-во зачисленных 3-го приоритета	0	23	4	0
Кол-во зачисленных 4-го приоритета	0	0	20	0

5.4. График динамики проходных баллов

ДИНАМИКА ПРОХОДНЫХ БАЛЛОВ

Динамика проходных баллов по дням



PDF-отчёт включает следующие разделы:

- Дата и время формирования отчёта
- Таблица проходных баллов по программам (или статус "НЕДОБОР")
- График динамики проходных баллов (при наличии данных за несколько дней)
- Списки зачисленных абитуриентов (ID + сумма баллов) по каждой программе
- Детальная статистика: количество заявлений и зачисленных по приоритетам

6. Ссылка на репозиторий и README

6.1. Репозиторий проекта

GitHub: <https://github.com/rawer470/AdmissionAnalysis>

6.2. Видеоролик

https://vkvideo.ru/video594499416_456239036?list=ln-2eQ47WIJvy1dzdIvef&t=1m13s